

Practical-8

BFS Traversal

Theory :-

A Binary Tree is a tree data structure in which each node can have at most two children: a left child and a right child.

In the given program, the binary tree is created recursively using the buildTree() function. The value -1 is used to indicate that there is no node at that position.

After constructing the tree, Breadth First Search (BFS) or Level Order Traversal is performed using a queue.

BFS visits the nodes level by level from left to right. Since a queue follows the FIFO (First In, First Out) principle, the root is processed first, followed by its children and then the next level.

Code :-

```
#include <iostream>
#include<queue>
using namespace std;

class node{
public:
    int data;
    node*left;
    node *right ;

    node(int data){
        this -> data = data;
        this -> left = NULL;
        this -> right = NULL;
    }
};

node* buildTree(node*root){
    cout << " Enter the Data  : ";
    int data ;
    cin >> data ;

    root = new node(data);

    if (data == -1){
```

```

        return NULL;
    }
    root = new node(data);

    cout << " for left node  \n";
    root -> left = buildTree(root->left);

    cout << "for right node  \n" ;
    root -> right = buildTree(root -> right );

    return root ;
}

void BFSTraversal(node*root){

    if (root == NULL)
        return;

    queue<node*>q;
    q.push(root);

    while (!q.empty()){
        node*temp = q.front();
        q.pop();

        cout << temp -> data  << "  ";

        if (temp -> left){
            q.push (temp -> left);
        }
        if (temp -> right){
            q.push(temp -> right);
        }
    }
}

int main (){

    node * root = NULL;
    root = buildTree(root);

    BFSTraversal(root);

    return 0 ;
}

```

Output :-

```
● PS C:\Users\shiva\Desktop\UNI Coding\DAA> cd "c:\Users\shiva\I
; if ($?) { .\Practical_8a_BFS_Traversal }
Enter the Data : 5
for left node
Enter the Data : 3
for left node
Enter the Data : -1
for right node
Enter the Data : -1
for right node
Enter the Data : 3
for left node
Enter the Data : -1
for right node
Enter the Data : -1
5 3 3
○ PS C:\Users\shiva\Desktop\UNI Coding\DAA> █
```

Analysis Of Algorithm :- Coin Change Combination

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$